SAPIENZA
UNIVERSITÀ DI ROMA

Evaluation of Statistical Methods for Generative Password-guessing models

Facoltà di Ingegneria dell'informazione, informatica e statistica
Informatica

Roberto Di Rosa
ID number 2043388

Advisor
Prof. De Gaspari

Co-Advisor

Academic Year 2025/2026

Evaluation of Statistical Methods for Generative Password-guessing models
Sapienza University of Rome

This thesis has been typeset by LaTeX and the Sapthesis class.

Author's email: dirosa.2043388@studenti.uniroma1.it

Abstract

"Don’t be overwise; fling yourself straight into life, without deliberation; don’t be afraid - the flood will bear you to the bank and set you safe on your feet again." Fyodor Dostoevsky

Contents

1 Introduction 1

2 Motivation 2

3 Background and Related Work 4
- 3.1 Monte Carlo Estimation . . . 4
- 3.2 Markov Chains . . . 4
- 3.3 Cython . . . 5
  - 3.3.1 Cython functions . . . 5
  - 3.3.2 Compilation and Distribution . . . 5
- 3.4 Recurrent Neural Networks(RNN) . . . 6
- 3.5 Long Short Term Memory(LSTM) . . . 6
  - 3.5.1 Binary Search . . . 7
- 3.6 Probabilistic Password Models . . . 7
- 3.7 Previous Works . . . 9
  - 3.7.1 Classical Works . . . 9
  - 3.7.2 Modern . . . 9

4 Methodology 10
- 4.1 Optimizing FLA sampling process . . . 10
  - 4.1.1 Pure Python . . . 10
  - 4.1.2 Heap-item . . . 10
  - 4.1.3 Custom-heap . . . 11

5 Experimental Setup 12
- 5.1 Dataset . . . 12
- 5.2 Model and Training . . . 12
- 5.3 Evaluation Metrics . . . 12
  - 5.3.1 Password Rank Estimation . . . 12
  - 5.3.2 Guess Number and Guessing Percentage . . . 12
  - 5.3.3 Estimation with Monte Carlo . . . 13
- 5.4 Implementation Details . . . 14
  - 5.4.1 Explanation . . . 14
  - 5.4.2 Caching step . . . 18

6 Results and Analysis 20
- 6.1 Figures . . . 20
- 6.2 Tables . . . 21

7 Conclusion 23

8 Appendix **24**
.1 UML 24
.2 Extended FOL 24

Bibliography **26**

Chapter 1

Introduction

In the field of probabilistic Password-guessing models there is a big problem, we take a long time to evaluate a passwords rank and a model. This is important since we want to give the maximum security possible to services relying on passwords(e.g. a website), and to do that we need fast and accurate password rank evaluation. We must address these because if not then the security of many services would be left compromised by this kind of attacks which rely on PGMs.

This thesis makes the following contributions:

0. A Cython-based item structure that reduces memory usage by 59.79%.
1. A custom-heap implementation that reduces memory usage by 87.47%.
2. Integration into MAYA to accelerate Monte Carlo estimation.
3. Evaluation of accuracy across Θ sizes.
4. A reproducible implementation.

Chapter 2

Motivation

Human-chosen passwords today face a grave danger since malicious actors have devoleped new methods, based on Password-guessing Models. We need to understand how these attackers prioritize guesses and how many attempts are needed to break a password using this "novel" models.

Therefore we developed Password-guessing Models of our own to study and to evaluate the strength of a password. They assign a probability to each password and, by ranking passwords in decreasing probability order, they allow us to approximate how quickly an adversary could guess them. This rank is the foundation of virtually all password-strength meters, risk assessments, and password-policy design.

However, computing password ranks is computationally expensive. For this reason, modern frameworks rely on Monte Carlo estimation [2], which approximates password rank using samples drawn from the model's distribution in an i.i.d manner. Monte Carlo estimation is effective, but it places a heavy computational burden on sampling, since millions of samples are required for stable accuracy.

This is critical for models such as Fast, Lean and Accurate (FLA), whose primary strength is client-side efficiency. FLA is specifically designed to run on low-resource devices, meaning that memory usage and sampling performance become decisive factors. The original implementation inside MAYA, while correct, suffers from two limitations:

High memory usage, due to Python objects (tuples, strings, floats) stored in a large heap.

These limitations create bottlenecks when evaluating models on large datasets or when computing Monte Carlo estimates for many passwords. For example, MAYA requires building a Monte Carlo curve for each checkpoint of each model. If sampling is slow or memory-intensive, evaluating large models becomes impractical, and researchers are unable to explore bigger Θ values or newer model architectures.

The goal of this thesis is to address these limitations by optimizing the sampling process at the core of FLA-based Monte Carlo estimation, and to see if montecarlo estimation for the evaluation of a model keeps up after 10^7 as previously proved by [4]. We introduce two new heap-based sampling implementations:

heap-item, a Cython-optimized drop-in replacement for Python heap items, reducing memory usage while keeping compatibility.

heapcy, a fully custom Cython heap storing only integer references to string positions, achieving substantial memory reduction.

These improvements are not merely engineering optimizations. They directly enable MAYA to scale to larger sample sizes, evaluate more complex models, and compute Monte Carlo estimates more efficiently. By reducing memory consumption

by up to 87%, our work makes Monte Carlo-based password evaluation faster, more accurate, and more accessible on standard hardware.

This allowed us to push the topk sampler to $5 * 10^8$ and prove that the Monte Carlo Estimation keeps up.

Chapter 3

Background and Related Work

3.1 Monte Carlo Estimation

Given an analytically complex problem to solve, using the Monte Carlo estimation will give us a value close enough to the real solution, even though it is much more simple analytically speaking. If we take a sample of n items from the main problem, we then perform a sum over the values then divide at the end with $\#samples$. This will give us an approximation of the expected value of our original problem. Let us say that we do not know the probability of a coin toss, we can use Monte Carlo to estimate this probability:

$$\text{Given a Bernoulli random variable } X \in \{0, 1\}$$

$$\hat{p} = \frac{1}{n} \sum_{i=0}^{n-1} X_i$$

As shown in the formula, above and according to the weak law of large numbers [1]

$$\lim_{n \to \infty} \frac{1}{n} \sum_{i=1}^{n} X_i = 0.5$$

3.2 Markov Chains

Markov chains give us a framework through which we can model sequential events it works like this: we have a set of states for each state we have associated a probability distribution of going to another, that can be easily represented with a matrix. To formalize this we can look at the below representation

$$\text{Let } S = \{S_1, S_2, \ldots, S_n\} \text{ be the state space.}$$
$$\text{Let } P \text{ be the } n \times n \text{ transition matrix, where}$$
$$P_{ij} = P(X_{t+1} = S_j \mid X_t = S_i)$$

We can see applications of this in fields such as: NLP[2], PPM[3] etc...

[1] The weak law of large numbers states that an observed sample average from a sample will be closer to the true population average as the sample grows larger. [3]

[2] Natural Language Processign

[3] Probabilistic Password Models

3.3 Cython

Cython (not to be confused with CPython) is a superset of Python. It is a compiled language, rather than interpreted, and allows for the creation of C/C++ extensions for Python with ease. The typical workflow is as follows: We first define compiler directives (starting with #). These tell the compiler how to translate the code to C/C++ and handle bindings. We then write the logic using Python-like syntax.

3.3.1 Cython functions

Cython provides three types of function declarators:

- **cdef**: Defines a C-only function. These are very fast but cannot be called from external Python code. By default, they act like standard Python functions and hold the GIL[4], allowing access to Python objects. However, if defined with `nogil` (e.g., `cdef void func() nogil:`), they release the GIL; in this mode, Python structures cannot be accessed.
- **cpdef**: Creates two versions of the function: a C version for use within Cython and a Python wrapper for external use. This offers the best of both worlds but introduces slight overhead compared to pure `cdef`.
- **def**: These are standard Python functions. They are accessible from anywhere but incur the standard Python function call overhead.

This distinction allows us to write performance-critical loops/operations in `cdef` functions while keeping the interface accessible via `cpdef` or `def`.

Caller	Can call cdef?	Can call cpdef?	Can call def?
Cython (cdef/cpdef)	Yes	Yes	Yes
External Python	No	Yes	Yes

Table 3.1. Calling Conventions: Who can call whom?

3.3.2 Compilation and Distribution

Cython's compiler is easily installed with pip, it will output .whl files that then can be imported into Python files like normal. The problem is that it can be burdensome to compile for every Python Version Header with only the Cython compiler. Therefore we opted to use cibuildwheel which when integrated with Github's CI/CD it makes distributing the software really easily.

[4]The Global Intepreter Lock is a Python feature which was introduced to elimimate race conditions in parallel operations on objects, Python needs it because objects are tracked by the Garbage collector by keeping count of the references of said object when the references reach 0 then the garbage collector frees memory, if there are parallel operations on the same object this could lead to an early free inducing a use after free, or race conditions

3.4 Recurrent Neural Networks(RNN)

Models that have 3 layers[5]:

0. Input Layer X
1. Hidden Layer H
2. Output Layer Y

This is what happens at each step t: the input vector x_t is used by the RNN to update its hidden state h_t, with this function[5]:

$$h_t = \sigma_h(W_{xh}x_t + W_{hh}h_{t-1} + b_h), \tag{3.1}$$

W_{xh} is the matrix that represents the weights between the input layer and the hidden one[5], W_{hh} is the weight matrix for the recurrent connection and b_h is just the bias vector.

At the end the model will update it is y_t state in this manner [5]:

$$y_t = \sigma_y(W_{hy}h_t + b_y) \tag{3.2}$$

W_{hy} is the matrix that represents the weights between the hidden layer and the output layer[5], and b_y is the bias vector. The recurring part is the fact that each time we are computing the hidden state, however this model type does not have a good "Memory", because during the hidden state calculation the gradient starts to "vanish" which means that the model cannot learn long sequences of words, to solve this problem we have many evolutions of this kind of mode.

3.5 Long Short Term Memory(LSTM)

This is an evolution of RNN's which mitigates the vanishing gradient problem by controlling it. They introduce the concept of these 3 gates:

0. input
1. forget
2. output

Each of these tell the model how much of the previous, input,hidden, and output states to consider from the previous state. Even though we have this improvement over the base model, this is still not perfect as the vanishing can still occur, nevertheless it is better than the normal one[5].

[5]This is a simplification [5]

3.5.1 Binary Search

Searching algorithm that runs in $O(log_2 n)$, it works on an sorted iterable like a list, tuple, etc... example: Let us play the guessing game, I want you to guess the number I am thinking of, this number is $n \in \mathbb{N}0 \leq n \leq 100$ and you start "1" and I say higher, "2" and I say higher, "3" and I say higher, at this point you understand that it can take up to 100 tries or at most $O(n)$ So you come up with a better idea, you start from the middle, "50" and I say higher, "75" and I say higher, "100" and I say you guessed correctly. As we can see you guessed correctly with just 3 tries, instead of 100, this respects our claim of $O(log_2 n)$, we can confidently say that this is a good alogorithm, the problem with it is that everything must be sorted the order does not really matter you just switch the conditions.

3.6 Probabilistic Password Models

P.P.Ms are a necessary part of a modern login security system they are needed to evaluate the strength of the user's password, since today's attackers also use these models to crack passwords more efficiently. When defending they evaluate the strengh of a given password by telling you how many attempts it would take to guess it this is known as the **Rank** of a password, the higher the rank the more secure the password. Attackers prefer to use these models instead of brute forcing,Brute forcing is a procedure in which we try every possible password to find the correct one e.g. "What is Roberto Di Rosa's Gmail password?"
Let's assume he used only characters from the english alphabet and that the password has a maximum length of 12 characters, so we know that there are 26 characters in the english alphabet let's start by the first:

0. a
1. b
2. c ...
3. aa
4. ab
5. ac

We will try until we reach our result, the problem with this approach is that we might have to try: $26^{12} \sim 95.43$ quadrilions times which for any pc is very computationally demanding. We could use optimized versions like John The Ripper , which fasten the brute forcing job but are still very slow compared to these models. These probabilistic models instead rely on something else, after training we take all of the tuples of

0. password
1. password's probability

and put them inside of a set Γ, [6] we can now find out the rank of a given password α.

[6] If the model we are working with is implicit, we don't have access to any distribution, therefore at first it's impossible

$$R(\alpha) = |\{\beta \mid \beta, p_\beta \in \Gamma \wedge p_\beta > p(\alpha)\}| \tag{3.3}$$

This will give us the rank of a password or more simply the amount of attempts it would take to guess it, the main problem from this approach is its data intensivity the set of strings can be really big, so to be faster we need to use Monte Carlo's estimation3.1, we start by defining a set Θ, it will contain a sample of strings taken from a i.i.d.[7] distribution Ψ based on the internal one of the model. Furthermore the larger the size of Θ the better the estimation as proved in [2], this will help us especially with Low-probability passwords since they have higher variance. In this paper we implemented the version found in [2] without any improvements as seen in [6] this has not impacted on the performance of the guess% estimation. To this day we have not found a way to better this variance, the latest attempts have yielded some results in lowering the variance therefore making montecarlo more accurate especially for Low-probability passwords, but they are not good enough yet. We can now Estimate the **Rank**, we define $\hat{R}(\alpha)$ as the estimation of the Rank for a given password α:

$$\hat{R}_\Psi(\alpha) = \hat{\mathbb{E}}_{\beta\sim\Psi}\left[\frac{\mathbb{1}[p(\beta) > p(\alpha)]}{p(\beta)}\right] = \frac{1}{n}\sum_{\beta\in\Theta}\frac{\mathbb{1}[p(\beta) > p(\alpha)]}{p(\beta)}[2] \tag{3.4}$$

This is a good estimation but if we need to estimate a lot of passwords, this will not be fast since each time we need to sample for montecarlo and calculate the rank, so the solution is to precalculate the rank and samples so that we do not need to do it multiple times, so let A be the descending sorted array of probabilities for getting generated by the model for each sampled password, and let C be the array of the estimated ranks for each password.

$$\text{Let } \mathbf{A} = [p(\beta_0), p(\beta_1) \ldots, p(\beta_n - 1)]$$
$$\text{Let } \mathbf{C}[i] = \frac{1}{n}\sum_{j=0}^{i} -1\frac{1}{A[j]} = C[i-1] + \frac{1}{n \cdot A[i]}$$

So given a password α to find its rank given A and C, we just need to reverse binary search 3.5.1 in A so that we find the index i of the probability which is higher than $p(\alpha)$, We than take the value stored at $C[i]$ which will be the $\hat{R}(\alpha)$, so we found our solution really fast. We measure the efficacy of the model by seeing in how many attempts it takes to guesse its test-dataset completely, the formula to do this is very simple:

$$G(x) = \frac{|\mathcal{S}(x) \cap \mathcal{T}|}{|\mathcal{T}|} \tag{3.5}$$

$x \in \mathbb{N}$ is the number of guesses the model makes,$\mathcal{T}$ is the set of all the test-dataset passwords, $\mathcal{S}(x)$ is the top-x sampling function that gives us the top-x most probable strings to be generated by the model. $G(x)$ is the ratio of the intersection between the sample set and the test-dataset,

They work by getting trained on a dataset(e.g. RockYou,Libero,TaoBao,etc...), from these datasets they will take the most frequent passwords and they will create

[7] Indepently,Identically Distributed we need this because if the distribution is not iid therefore we will not be able to get accurate estimates

distributions for these passwords in a way as to reflect the frequency of characters all of this using one of these RNN's,Markov chains, PCFGs,LSTMS,Random Forests[7], They can be separated into 2 types:

0. **Trawling** are aimed guessing correctly as many passwords as possible[8] and are constrained just by the amount of resources and the number of guesses itself.

1. **Targeted** are aimed at guessing the password of a single user given some specifics (e.g.: Name,Surname,Sex,Nationality,etc...)

3.7 Previous Works

3.7.1 Classical Works

Fast Accurate And Lean(FLA)

FLA is an LSTM model that is used to guess passwords and study how effective an attacker might be at cracking passwords. As evidenced in [4] the model is very good at guessing passwords compared to PCFGs, Markov Models and more heuristic methods like John The ripper. Although in recent years there has been a development in to password guessing, as in the emergence of new kinds of model like: RFGuess [7], PASS LLM [8] etc... The authors of FLA have done an excellent job at making it very "lean" so that it can run easily on the client side, making it a really good model for password strength metering on the client-side which was a novel idea at the time. They also used Monte Carlo estimation[2] to efficiently look at how many guesses it would take to guess all of the test dataset [4], and proved reliability up to 10^7.

3.7.2 Modern

Password Guessing With LLMs

As seen in [8] they brought forward the concept of using LLMs for password guessing and obtained really good results in the evaluation of guesses. These LLMs can get to higher guessing numbers much faster than previous implementations, instead of

Chapter 4

Methodology

4.1 Optimizing FLA sampling process

In this section, we start by analyzing the problems we encountered with MAYA and discussing the solutions we found to the said problems, their strengths, and weaknesses.

The problem we encountered was the need to sample the top-k elements from the distribution within FLA. Solution 1: Implement the item in Cython to improve performance and reduce memory usage. Solution 2: Write all the heap in Cython and do not store any strings.

4.1.1 Pure Python

The first problem we tackled was the original implementation of FLA's sampling algorithm[4], which occupied a significant memory usage due to the usage of pure Python structures such as lists, tuples, strings, and floats. The main problem came from the fact that each item was saved as a tuple of str, float, which was appended into a list, and that list was manipulated with the heapq module. Figure 1 illustrates the significant memory consumption caused by the original technique. Python heap structures store each heap item as a fully-fledged Python object (tuple of string and float). This leads to high memory overhead due to:

0. object headers,
1. reference counting,
2. heap fragmentation,
3. garbage collector metadata,
4. repeated storage of identical strings.

As a result, storing 10^6 heap entries may require hundreds of megabytes, when combined with all of the other objects we keep in memory this results in really big memory usage in the magnitude of 200GBs when sampling $5 * 10^8$ making it impossible to scale Θ to the sizes required for big model Evaluation.

4.1.2 Heap-item

The first proposed solution was to build a heap-item custom module written in Cython thus maximizing performance and compatibility with Python. The main

problem with this approach is the fact that like in pure python, each heap item is an object that must be tracked by python's garbage collector, nevertheless this approach helped us reduce memory consumption by $\sim 59.79\%$ instead of the pure python approach as seen in Figure 1, alas further optimization was necessary, so we opted for a custom heap called heapcy.

Using Cython allows us to replace Python objects with C-structured objects giving us improved memory usage while also giving us ease of integration in python, we could have used c++ and then have written the integrations with nanobind, but that would have taken considerably more time and the fast software distribution would have been more complicated. Since we use cython we have access to cibuildwheel which when integrated with github CI/CD allows us to publish binaries to pypy rendering software distribution really easy. These Cython objects:

- are allocated in contiguous memory,
- avoid Python reference counting,
- do not trigger garbage collection,
- store fields compactly (the struct is made using chars and doubles which is much smaller than normal python strings, and floats)

This reduces per-item memory usage tremendously. However, each item is still an python object, so total memory still scales linearly with the number of stored items.

4.1.3 Custom-heap

Rather than simply implementing a heap in Cython, we adopted a different approach by storing only the string's position in a file as a reference. This method significantly reduced memory usage, as shown in Figure 1, and required only a single heap object instead of multiple instances as in previous implementations 4.1.1 and 4.1.2. This approach has given us an improvement of $\sim 87.47\%$ over 4.1.1 and a $\sim 68.85\%$ over 4.1.2 While this solution offers superior performance, it is more complex to use compared to the previous solutions, so the choice should depend on the specific requirements of your situation.

Since we don't actually need strings in memory during the sampling process, we can just write a password once to a binary file, and the heap can store only its integer file offset together with the probability value.

This design:

- avoids storing millions of Python strings,
- eliminates repeated allocations,
- stores heap entries as compact C structs,
- ensures that the heap consists of a single contiguous memory region,

The tradeoff is increased implementation complexity and the need for careful file handling, but the performance benefit is decisive.

Chapter 5

Experimental Setup

5.1 Dataset

For this paper we chose the RockYou dataset the same as in MAYA [1](split 80;20), for time reasons we couldn't test with other datasets. RockYou is the standard dataset used in nearly all password-guessing literature. Using it ensures comparability with previous work and aligns with the datasets used by MAYA and FLA. Its size and structure also provide statistically meaningful evaluation for both trawling and targeted password-guessing attacks.

5.2 Model and Training

As discussed, there was no time to implement the functions required to test with other models. Model wise we used FLA found in MAYA[1], and optimized as said in 4.1. The passwords were generated with a maximum length of 12 characters, the model was pre-trained in the RockYou dataset and as a test-config we used rq1.

To evaluate these models we need to see how accurate their guessing [4] is, the following section will show us how to do this in multiple ways.

5.3 Evaluation Metrics

5.3.1 Password Rank Estimation

Why can't we just use the deterministic rank ? While using the deterministic rank would give us the most precise result, it requires an enormous amount of reseouces since we first need fo to calculate the entire set of passwords yieldable by the model, so we opt for the estimation As described above, to get our guessing percentage estimate we first need to implement the Monte Carlo seen in [[2]]. To do this we need 1 set:

$\Theta := \{(pwd, prob) \mid (pwd, prob) \in \text{getTheta(model,sampleSize)}\}$ our Monte Carlo sample.

5.3.2 Guess Number and Guessing Percentage

The standard way to get the guessing percentage of a model is the empirical way:

Eq. Empirical guess number for a given x.

$$G(x) = \frac{|\mathcal{G}_x \cap \mathcal{T}|}{|\mathcal{T}|} \tag{5.1}$$

- $\mathcal{G}_x$ is the set containing the top-x sampling,
- $\mathcal{T}$ is the test-dataset containing all of the strings in the test-dataset.

This division give us the real ratio of how many passwords of the test-dataset where guessed which can be seen as a percentage in 2 last column. The former approach was very slow, we needed to find another way to reach a good enough solution.

5.3.3 Estimation with Monte Carlo

We employ a Monte Carlo estimate (5.2) to get a result similar to (5.1). $\hat{R}$, represents the estimate of the rank (number of times the model needs to guess[1]) to generate the password.

Thanks to this we require much less effort compared to the deterministic way, since all we need to do is to compute the Monte Carlo curve just once per Model Checkpoint, save it to .csv file and then for each guess number apply the formula above and we will have our results significantly faster than the previous approach.

The former approach was very slow since we needed to take the top_k where k is the guess number [1].

There are two ways to calculate the Monte Carlo estimation both specified in [2] and here included:

Eq. Monte Carlo Estimation for the rank of a password.

$$\hat{R}_\Psi(\alpha) = \hat{\mathbb{E}}_{\beta\sim\Psi}\left[\frac{1[p(\beta) > p(\alpha)]}{p(\beta)}\right] = \frac{1}{n}\sum_{\beta\in\Theta}\frac{1[p(\beta) > p(\alpha)]}{p(\beta)} \tag{5.2}$$

Array sorted in descending order

$$\text{Let } \mathbf{A} = [p(\beta_1), p(\beta_2)\ldots, p(\beta_n)] \tag{5.3}$$

Eq. Optimized version

$$\text{Let } \mathbf{C}[i] = \frac{1}{n}\sum_{j=1}^{i}\frac{1}{A[j]} = C[i-1] + \frac{1}{n\cdot A[i]}$$

Estimation of the guess number for a given x.

$$\hat{G}(x) = \frac{1}{|T|}\sum 1_{[\hat{R}(\alpha)\leq x]} \tag{5.4}$$

Eq. Variant of (5.4)

$$\hat{G}(x) = \frac{|\{\alpha \in \mathcal{T} \mid \widehat{R}(\alpha) \leq x\}|}{|\mathcal{T}|} \tag{5.5}$$

[1] As defined in [2]

5.4 Implementation Details

5.4.1 Explanation

get_theta

The implementation for FLA, is very simple we first call the get_theta function inside of which we instantiate a guesser objectm we then call the OneBatchToControlThemAll which will sample in batches from the model distribution calling IIDSampleBatched and then these samples are written to a csv file which is our montecarlo_sampling or Θ.

Algorithm 1: Hierarchical Password Generation Process

function GetTheta $(path, \mathcal{E})$;
Input : Output directory *path*, Config dictionary $\mathcal{E}$
`// 1. Factory method creates the Guesser object`
$guesser \leftarrow$ `GuesserBuild(`$\mathcal{E}$`);`
`// 2. The object executes the batch processing`
$N \leftarrow$ self.$n\_samples$;
$guesser$`.OneBatchToControlThemAll(`N, ***true***, $path$`);`

function OneBatchToControlThemAll $(n, path)$;
Input : Total samples n, Output path *path*
$BatchSize \leftarrow 2048$;
$generated \leftarrow 0$;
while $generated < n$ **do**
    $need \leftarrow \min(BatchSize, n - generated)$;
    `// Delegate math to the core sampler`
    $batch\_data \leftarrow$ `IIDSampleBatched(`$need$`);`
    Append $batch\_data$ to CSV at *path*;
    $generated \leftarrow generated + need$;
    if $generated \pmod{10} = 0$ **then**
        Print progress;

Algorithm 2: Core Monte Carlo Sampler

function IIDSampleBatched (n);
Input : Batch size n
Output : List of $(password, probability)$ tuples
$S \leftarrow$ array of n empty strings;
$L \leftarrow \vec{0}_n$; `// Log-probability accumulator`
$Finished \leftarrow \vec{\text{False}}_n$;
for $t \leftarrow 0$ **to** $MaxLen$ **do**
  if ***all*** $Finished$ **then**
    break;
  $X \leftarrow \text{Encode}(S)$;
  $P \leftarrow \text{Model}(X)$; `// Raw probabilities`
  `// Apply structural constraints`
  $M \leftarrow \text{GetValidNextMask}(S)$;
  $P \leftarrow P \odot M$;
  Normalize rows of P to sum to 1.0;
  if $t = MaxLen - 1$ **then**
    Force P to select END token;
  $next\_tokens \sim \text{Categorical}(P)$;
  for $i \leftarrow 0$ **to** $n - 1$ **do**
    if ***not*** $Finished[i]$ **then**
      $k \leftarrow next\_tokens[i]$;
      $L[i] \leftarrow L[i] + \ln(P[i, k])$;
      if $k = END$ **then**
        $Finished[i] \leftarrow$ **true**;
      else
        $S[i] \leftarrow S[i] + \text{Decode}(k)$;
return $\text{zip}(S, \exp(L))$;

_build_mc_curve

here we call the function _build_mc_curve which will build the mc-curve from the file saved by algorithm 1 it will return two arrays, A and C, A will be the array containing the probabilities in descending order and C will be the array containing the cumulative sum of the estimated ranks, such that for each probability in A at index $i \in [0..n] \wedge n \in \mathbb{N}$ the estimated rank of said probability will be at $C[i]$ If the arrays were not saved as tensors they will be saved so that future runs may be even faster.

Algorithm 3: Orchestration logic for building the Monte Carlo Curve

function BuildMCCurve ($\mathcal{E}$);
Input : Evaluation Dictionary $\mathcal{E}$
Output : Tensors A (Probabilities) and C (Guess Numbers)
`// Define paths based on base directory`
$P_{raw} \leftarrow$ "mc_samples.gz"; $P_{curve} \leftarrow$ "mc_rank_curve.gz";
$P_A \leftarrow$ "A.pt"; $P_C \leftarrow$ "C.pt";
if *files* P_A *and* P_C *exist* **then**
  `// Fast Path: Load pre-computed tensors`
  $A \leftarrow \text{Load}(P_A)$; $C \leftarrow \text{Load}(P_C)$;
else
  `// Check if raw sampling is required`
  if P_{raw} *does not exist* **then**
    `MonteCarloSampling(`$P_{raw}, \mathcal{E}$`);`
  `// Compute statistics from raw sample file`
  $(A, C) \leftarrow$ `ComputeCurveStats(`P_{raw}`);`
  `// Cache results for future runs`
  $\text{Save}(A \rightarrow P_A)$; $\text{Save}(C \rightarrow P_C)$;
  if P_{curve} *does not exist* **then**
    `SaveCurve(`P_{curve}, A, C`);`
return (A, C);

Algorithm 4: Calculation of statistical tensors from raw samples

function ComputeCurveStats (*path*);
Input : Path to raw Monte Carlo samples
Output : Tensor A (sorted probs), Tensor C (cumulative cost)
`// 1. Parse probabilities from file`
$Probs \leftarrow$ Read column 1 from CSV at *path*;
$A \leftarrow \text{Tensor}(Probs)$;
`// 2. Sort descending (most likely first)`
$A \leftarrow \text{SortDescending}(A)$;
`// 3. Compute expected guesses (CCS'15 Section 3.2)`
$N \leftarrow \text{Length}(A)$;
`// Compute inverse probabilities`
$C \leftarrow 1 \oslash A$;
`// Update C with cumulative sum:` $C_i = \sum_{j=0}^{i}(1/A_j)$
$C \leftarrow \text{CumSum}(C)$;
`// Divide by n so we finish applying montecarlo`
$C \leftarrow C \oslash N$;
return (A, C);

Algorithm 5: Serializes the computed curve to disk

function SaveCurve $(path, A, C)$;
Input : Output path, Tensors A and C
Open file at *path* for writing;
foreach *index i in A* **do**
    `// Write tuple (probability, expected_guesses)`
    Write row $(A[i], C[i])$ to file;

tester

Algorithm 6: Orchestrator for Monte Carlo Testing Session

function StartMonteCarloTest $(Name_{ckpt})$;
Input : Name of the checkpoint $Name_{ckpt}$
`// 1. Model State Management`
$Path_{file} \leftarrow$ JoinPaths$(Dir_{ckpt}, Name_{ckpt})$;
$Loaded \leftarrow$ `LoadCheckpoint(`$Path_{file}$`)`;
if *not* *Loaded* **then**
    Print "Checkpoint missing, starting training...";
    `StartTrain(`$Name_{ckpt}$`)`;
    `LoadCheckpoint(`$Path_{file}$`)`;
Print "Checkpoint loaded successfully.";
`// 2. Prepare Baseline Samples (Top-K)`
$P_{topk} \leftarrow$ "topk_guesses_str_float.gz";
$\mathcal{E} \leftarrow$ InitEval$(N_{samples})$;
if *File at* P_{topk} *does not exist* **then**
    Ensure directory exists;
    $Generator \leftarrow$ `SampleModel(`$0, \mathcal{E}$`)`;
    Open P_{topk} for writing;
    foreach $(pwd, prob)$ *in Generator* **do**
        Write "*pwd prob*" to P_{topk};
    Verify total lines written equals $N_{samples}$;
    `PostSampling(`$\mathcal{E}$`)`;
`// 3. Select Iterator Source and Execute`
if *Config.TestTopK* ***is true*** **then**
    `// Use the Top-K file generated above`
    $Iterator \leftarrow$ `GetGeneratorFromTopK(`$P_{topk}, Device$`)`;
else
    `// Use the full test dataset (default)`
    $Iterator \leftarrow$ `GetTestDataset()`;
$t_{start} \leftarrow$ Time();
`MonteCarloTest(`$\mathcal{E}, Iterator$`)`;
$t_{delta} \leftarrow$ Time() $- t_{start}$;
Print "Test completed in: " t_{delta};

Algorithm 7: Rank Estimation via Monte Carlo Curve

function MonteCarloTest $(\mathcal{E}, \mathcal{T})$;
Input : Config $\mathcal{E}$, Test Iterator $\mathcal{T}$ (pairs of $pwd, prob$)
Output : CSV file with estimated ranks $\hat{r}$
`// 1. Retrieve statistical curve tensors`
$(A, C) \leftarrow$ `BuildMCCurve(`$\mathcal{E}$`)`;
Open output file for writing;
foreach $(pwd, target\_prob)$ *in* $\mathcal{T}$ **do**
    `// Find where target_prob fits in distribution A`
    `// Since A is descending, we search for insertion point`
    $idx \leftarrow$ `BinarySearch(`$A, target\_prob$`)` - 1;
    if $idx < 0$ **then**
        $\hat{r} \leftarrow 0$;
    else
        $\hat{r} \leftarrow C[idx]$;
    Write $(pwd, \hat{r}, target\_prob)$ to file;

5.4.2 Caching step

To test the estimation for the evaluation we need to first take the test-dataset and for each password we need to get the probability of it being generated by the model and for good purpouses we should also sort all of it. to do that we employ our heapcy which makes this process much less memory intesive than it needs to be. we than write everything to a compressed file, if so that we do not need to generate it every time, the problem stems from the fact that for each time the model is trained we need to do this generation since the model could have changed its internal distribution, to optimize this we could work in batches to get the probability of multiple passwords at the same time, and to write into a file. same with the heap, we could maybe work with multiple heaps and then merge them togheter into one, when finished, all of course using batches and the gpu.

Algorithm 8: Cached Test Dataset Retrieval

function GetTestDataset ();
Output : Stream of $(pwd, prob)$ tuples
$Path \leftarrow$ "test_dataset.csv.gz";
if *File at Path does not exist* **then**
    `// Expensive generation step`
    $Stream \leftarrow$ `GenerateSortedTestSet()`;
    Open $Path$ for writing (compressed);
    while *Stream is not empty* **do**
        Write batch from $Stream$ to $Path$;
`// Yield from the cached file`
return Stream reading from $Path$;

Algorithm 9: External Sort for Probability Dataset

```
1  function GenerateSortedTestSet ();
   Input  : List of test passwords P_test
   Output : Generator yielding passwords sorted by probability
2  H ← New Min-Heap;
3  Tmp ← Create temporary binary file;
   // Phase 1: Calculate and Index
4  foreach pwd in P_test do
5      if pwd is valid then
6          offset ← Tmp.tell();
7          prob ← GetProbability(pwd);
           // Store raw data on disk
8          Write (pwd, prob) to Tmp;
           // Store index in RAM (prob, file_offset)
9          H.Push((prob, offset));
   // Phase 2: Sort and Retrieve
10 SortedOffsets ← Extract N largest from H;
11 Clear H from memory;
12 foreach offset in SortedOffsets do
13     Seek Tmp to offset;
14     Read (pwd, prob) from Tmp;
15     yield (pwd, prob);
   // Cleanup runs in finally block
16 Delete Tmp;
```

Chapter 6

Results and Analysis

6.1 Figures

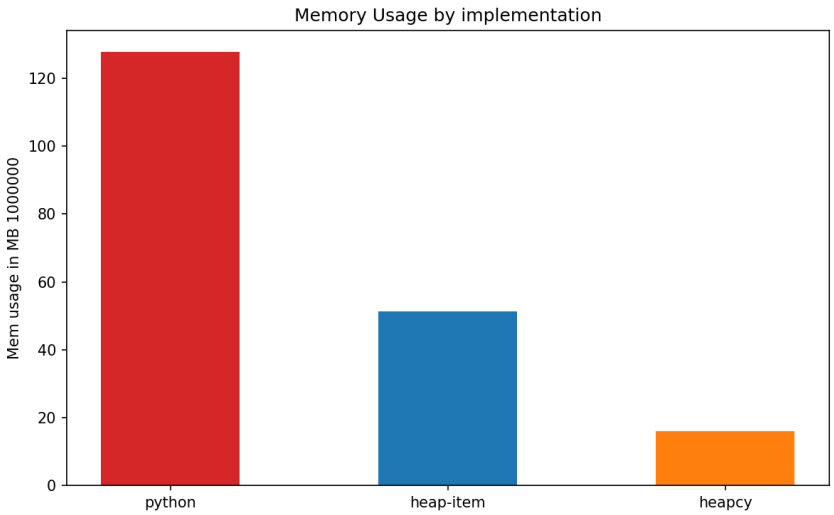
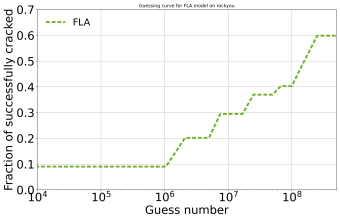
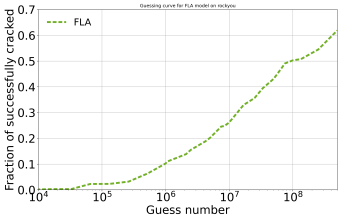


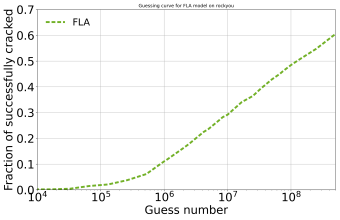
Figure 1. Memory usage of the three heap implementations when storing 10^6 items.



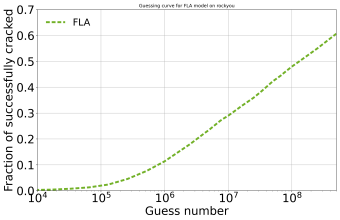
(a) Guess % with a $|\Theta| = 10^1$



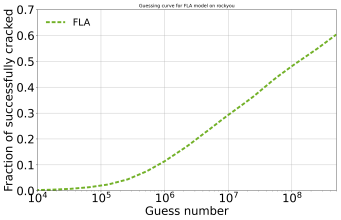
(b) Guess % with a $|\Theta| = 10^2$



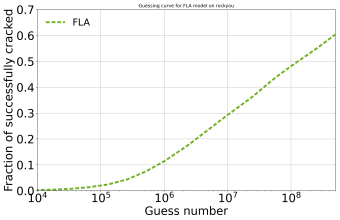
(c) Guess % with a $|\Theta| = 10^3$



(d) Guess % with a $|\Theta| = 10^4$



(e) Guess % with a $|\Theta| = 10^5$



(f) Guess % with a $|\Theta| = 10^6$

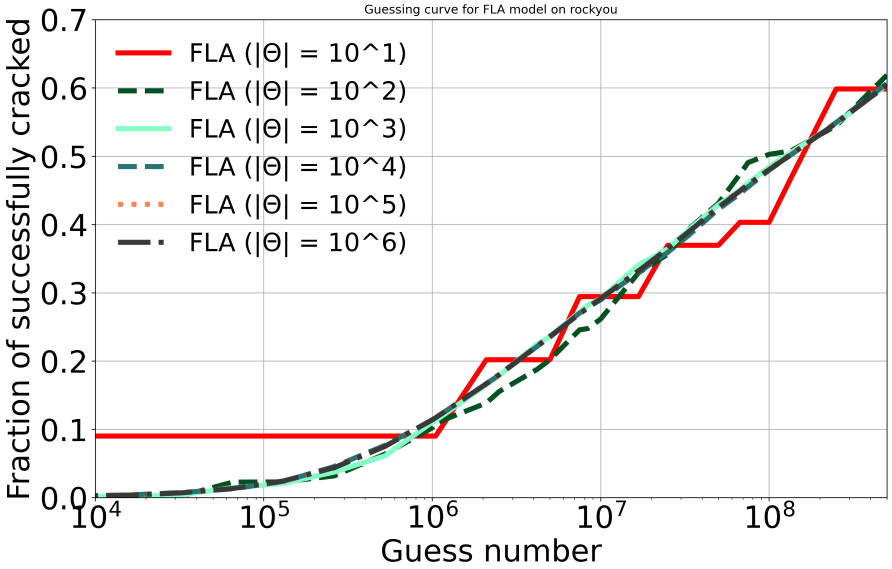


Figure 4. Comparison of the montecarlo run for every theta size

6.2 Tables

#Item	Pure Python	Heap Item	Heapcy
10^6	127.72	51.36	16.00
10^5	12.77	5.13	1.60

Table 1. Size occupied in MB

As can be seen the gains have the same rate at any amount of items, so by using 10x the amount of items we will have 10x the amount of memory used.

Guess #	Rate 10^1	Rate 10^2	Rate 10^3	Rate 10^4	Rate 10^5	Rate 10^6	MAYA %
$1.0 \cdot 10^6$	9.01%	10.22%	11.03%	11.35%	11.37%	11.40%	11.40%
$2.5 \cdot 10^6$	20.19%	15.54%	17.91%	17.95%	17.97%	18.02%	18.04%
$5.0 \cdot 10^6$	20.19%	20.19%	23.60%	23.49%	23.57%	23.61%	23.66%
$7.5 \cdot 10^6$	29.43%	24.57%	27.17%	27.02%	26.94%	26.90%	26.94%
$1.0 \cdot 10^7$	29.43%	26.15.%	29.15%	28.97%	29.25%	29.16%	29.20%
$2.5 \cdot 10^7$	36.95%	35.63%	36.41%	35.91%	36.42%	36.45%	36.48%
$5.0 \cdot 10^7$	36.95%	43.09%	42.88%	42.23%	42.59%	42.47%	42.49%
$7.5 \cdot 10^7$	40.31%	49.05%	46.09%	45.50%	45.84%	45.80%	45.82%
$1.0 \cdot 10^8$	40.31%	50.26%	48.43%	47.95%	48.11%	48.02%	48.04%
$2.5 \cdot 10^8$	59.84%	54.45%	54.74%	54.92%	54.85%	54.83%	54.85%
$5.0 \cdot 10^8$	59.84%	61.87%	60.45%	60.65%	60.35%	60.42%	60.47%

Table 2. Comparison of crack rates over five runs with different θ size.

Chapter 7

Conclusion

In Conclusion we managed to optimize the sampling of FLA, added montecarlo to MAYA and we proved with results how the montecarlo estimation is very good at estimating the evaluation. We went beyond FLAs original paper by proving the result up to $5.0 \cdot 10^8$ this may imply that we do not need to the evaluation the good old way, by using enumeration, instead we can use montecarlo to see how good a model really is. Being able to extend Monte Carlo sampling to $|\Theta| = 5 \cdot 10^8$ is important, since the accuracy of it is entirely dependent on the size of $|\Theta|$ The larger the sample size the smaller the variance in the tail of the distribution(Low probability passwords), which is where many real passwords reside. Our optimizations therefore enable a better evaluation of password-guessing models without requiring enormous amounts of memory usage. In future works we should expand this method to every model implemented inside of MAYA, and try montecarlo for even larger numbers, since the new optimization allows us to enumerate up to even bigger numbers without encurring into big performance issues. We should as well optimize the caching part 9, since for now it is all sequential and takes a considerable amount of time $\sim 4hr$ for RockYou test-dataset $\sim 4.0 * 10^6$ passwords. We could also improve the heapcy implementation to have a dynamic memory instead of the current implementation.

Chapter 8

Appendix

.1 UML

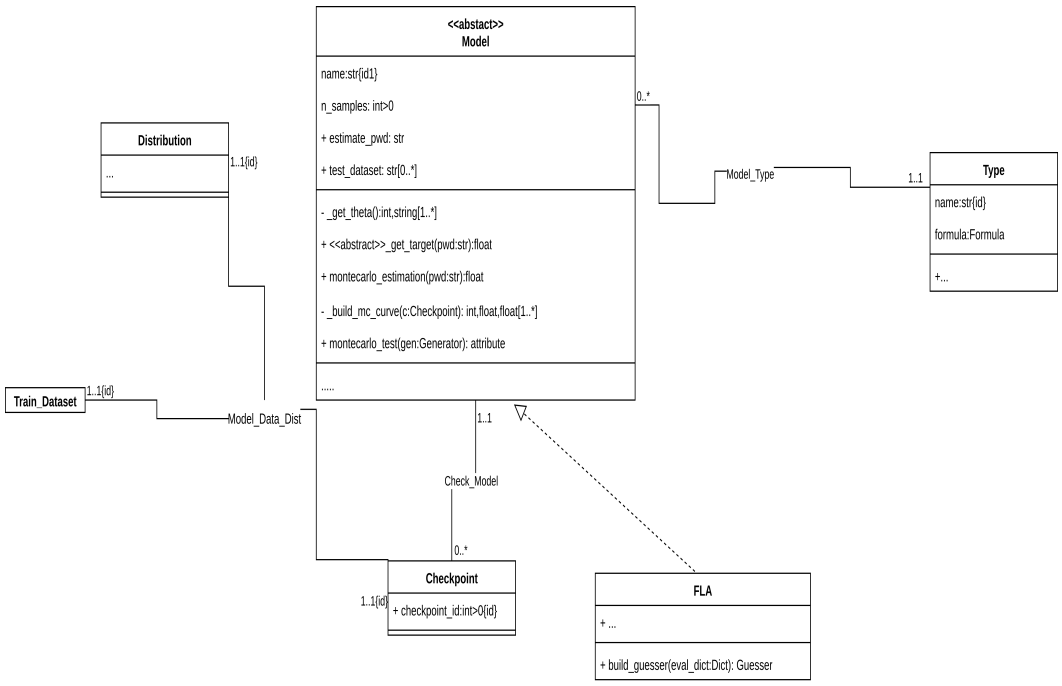


Figure .1. Analysis

.2 Extended FOL

$$\text{Let } \mathbf{A} := \{(i, prob) \mid i \in \mathbb{N} \wedge \ (x, prob) \in sorted(\{x, p \mid \exists pwd \in \Theta \ \wedge x \in \mathbb{N} \wedge P(pwd, p) \ \wedge \ x = x + 1\}, sort\_func) \wedge i = |\mathbf{A}| + 1\} \quad (5.3)$$

$$
\begin{aligned}
&sort\_func(t_1 : int \geq 0, real(0..1), t_2 : int \geq 0, real(0..1))) \rightarrow Bool : \\
&\quad\quad preconditions : None \\
&\quad\quad postconditions : \\
&\quad\quad |ret := \forall\ n_1, n_2, \\
&\quad\quad |Real(t_1, n_1) \wedge Real(t_2, n_2) \implies n_1 > n_2 \\
&\quad\quad |\text{return ret}
\end{aligned}
$$

$$
\text{Let } \mathbf{C} := \{(i, n) \mid \exists x, prob \wedge i \in \mathbb{N} \wedge n \in \mathbb{R}_{>0} \wedge |A| > 0 \wedge n = \frac{1}{|A|} \cdot \sum_{x, prob \in A}^{i \leq x} \frac{1}{prob}\}
$$

Bibliography

[1] Corrias, W., De Gaspari, F., Hitaj, D., and Mancini, L. V. MAYA: A unified benchmarking framework for generative password-guessing models. In *2026 IEEE Symposium on Security and Privacy (SP)* (2026). ArXiv preprint arXiv:2504.16651v2. Available from: https://arxiv.org/abs/2504.16651v2.

[2] Dell'Amico, M. and Filippone, M. Monte carlo strength evaluation: Fast and reliable password checking. In *Proceedings of the 22nd ACM Conference on Computer and Communications Security (CCS'15)*. ACM Press (2015). Available from: https://dl.acm.org/doi/pdf/10.1145/2810103.2813631.

[3] Loève, M. *Probability Theory I*. Springer, New York, 4th edn. (1977). Available from: https://archive.org/details/probabilitytheor0000loev_h6n0/mode/2up.

[4] Melicher, W., Ur, B., Segreti, S. M., Komanduri, S., Bauer, L., Christin, N., and Cranor, L. F. Fast, accurate, and lean: A low-overhead, high-accuracy method for a-priori password strength estimation. In *25th USENIX Security Symposium (USENIX Security 16)* (2016). Available from: https://www.usenix.org/conference/usenixsecurity16/technical-sessions/presentation/melicher.

[5] Mienye, I. D., Swart, T. G., and Obaido, G. Recurrent neural networks: A comprehensive review of architectures, variants, and applications. *Information*, **15** (2024). Available from: https://www.mdpi.com/2078-2489/15/9/517.

[6] Stanek, M. Revisiting monte carlo strength evaluation. *arXiv preprint arXiv:2408.00124*, (2024). Available from: https://arxiv.org/abs/2408.00124v1, arXiv:2408.00124.

[7] Wang, D., Zou, Y., Zhang, Z., and Xiu, K. Password guessing using random forest. In *32nd USENIX Security Symposium (USENIX Security 23)* (2023). Available from: https://www.usenix.org/conference/usenixsecurity23/presentation/wang-ding-password-guessing.

[8] Zou, Y., An, M., and Wang, D. Password guessing using large language models. In *USENIX Security Symposium (USENIX Security 25)* (2025). Available from: https://www.usenix.org/conference/usenixsecurity25/presentation/zou-yunkai.